Game Dev Challenges and Programming

Basic Concepts

Assets & Pipelining

The collective entirety of a game's playable "world" alternatively called **gameplay universe** by my wiser dictionary, is ultimately made of some basic 3D or 2D "things","stuff" and objects and a set of rules that define how they are to behave in the gameworld.

Assets

Mesh

An actual “physical” object in the world.

Textures

Are 2D images used for different purposes. There are different variations of textures defining an equally necessary aspect of for it’s mapping.

Albedo Map / Diffuse Map

Is the image itself.

Normal Map

Stores surface direction xyz in RGB.

Used for fake lighting detail

Mask Map / Control Map / Alpha Mask/ Opacity Mask / Roughness Mask / Gloss Mask / Specular Mask

Controls where and how effects apply.

Materials

These are shader setups defining how to take textures and simulate. They are instructions for rendering data and are mainly concerned about how to render things on “surfaces”.

Material Instances

Tweaked version of material still reliant on the base version. e.g: We have a Sky material then we create a morning on outta it

Cell

Game worlds are purposefully divided into chunks for manageability upon load. These chunks are called Levels/Cells/Worlds in Game Engine system view.
From open world games to linear ones they all use this system but in open world ones typically the world loading should happen by more estimations and according to variables such as draw distance and player location to keep the open world from being loaded late.

Level/UMap

A saved definition of a noticeably large world section.

UWorld

A runtime simulation instance created from one or more umap defined levels.

The engine could run/bind to only one UWorld at a time.

Pipelining

Coordinate Systems

UV Coordinate System

Shader Node Graphs

Abstractions for basic functions involved in shading.

Graphics

Lights

Global Illumination

Light Maps (old solution)

RTGI (new solution)

Reflections

Generally there are 2 main reflection models:

1- Blinn Phong
2- Physically Based Rendering/Microfacet BRDF

- In the Blinn Phong model:

Final Color = Diffuse + Specular Highlight

- In the modern PBR instead we have:

Base Color,Metallic, Roughness

Texture Cubes (Blinn Phong)

Used static textures as reflections.

Screen Space Reflections

Dynamic reflections cast.

Stencil

Culling

Technical Art Concepts

Sky

Previously implemented as a dome mesh with materials defining what was rendered on it but new procedural methods have emerged in UE5 for per pixel rendering such as the atmosphere role.

Sky Material

The implementation of sky using material is done through:

1- Implementing a gradient: Using texture coordinates.

Adhoc

Rendering

Optimization Aliasing

Reduction

Aliasing happens as the details get too small for the pixels to be rendered well enough. The reason in many older games this wasn't as big of a deal was with how the engines handled 3D objects through decreasing the LOD / Using a decreased LOD version of objects after a distance back in the days of DL1 and such.

3D Modelling

Optimization Technique

Typically the 3D models use rather simple meshes for hair and then proceed to implement the hair thru textures transparency to give the illusion of empty pots in hair.

UDK (Older Approaches)

Upon running UDK it loads the core modules and Engine packages (some basic default template) upk files which are precompiled asset libraries mounted at runtime.

In UDK sky is a dome, like an actual 3D mesh as a dome

Material Editor

The material editor actually builds the GPU shader graph for materials.

It uses a chain of nodes to achieve it's results. This chains of nodes is made of:

- Nodes:basic building blcoks of material graphs.

Within the material editor we use **ctrl + click** to drag the nodes close and away to each other.

In order to connect nodes to one another we click on their pins. In order to cut connection among nodes we use Alt + Click

Nodes

Each node could be from:

1- **The material function library:**
2- **Material Expression:**

Either way the input pins of a node are on the right side, with the output pins on the left side.

Material Output Nodes

These are the final interface between graph and the GPU shader result. Everything built in graph must ultimately be plugged into them. It defines the values the generated shader must compute.

- Every material must have ONE single output node.

- The output node is always there by default with a pretty descriptive name like Material or PreviewMaterial_2.

- The output node does not have an output pin cause… it IS the output pin itself.

The output node has the following input pins:

1- Diffuse: Takes color, constant3vector.
2- DiffusePower:
3- Emissive:
4- Specular: Strength of light reflection effect, takes a number.
5- SpecularPower: The amount of area of reflection dividee.
6- Opacity:
7- OpacityMask:
8- Distortion:
9- TransmissionMask:
1- TransmissionColor:
2- Normal:
3- CustomLighting:
4- CustomLightingDiffuse:
5- AnisotropicDirection:
6- WorldPositionOffset:

Material Expressions

They are all the non output nodes that we add.

Constant3Vector

It represents a color.

TextureCoord

Component Mask

Takes a component ed value and treats it’s first 4 component as RGBA, with Component Mask allowing you to mask off the components that you don’t like using it. All components except the ticked ones will be masked off.

Texture Editor

Typically we define textures in the following resolutions

256x256 or 512x512 (Small Props)

Cans,books,etc.

512x512 or 1024x1024 (Medium Props)

Crates, weapons, furniture.

1024x1024 or 2048x2048 (Hero Assets)

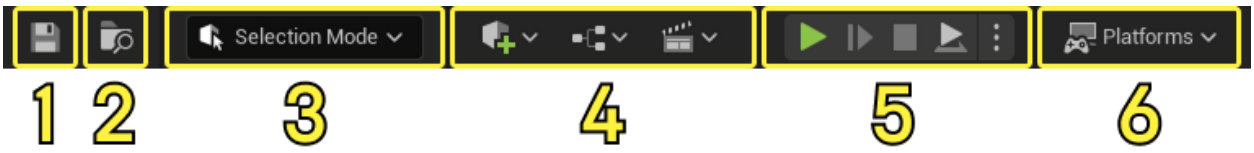
Player weapons,important characters.

Unreal Engine 5.7 Documentation

1 Menu Bar	Uses these menus to access common application actions, like saving, and creating new levels. You'll also find options for opening editor windows and tools that are useful for specific functions like debugging and more.

2	Main Toolbar	Contains shortcuts for some of the most common tools and editors in Unreal Engine, as well as shortcuts to enter Play mode (run your game inside Unreal Editor) and to deploy your project to other platforms.
3	Viewport Toolbar	Includes common tools used to manipulate objects in the level by moving, rotating, and scaling them, as well as snapping tools for moving objects along a grid, rotation angle, or scaling amount. It also includes perspective and orthographic views along with debugging and visualization view mode and other settings you can use while working in the viewport.
4	Level Viewport	Displays the contents of your Level, such as Cameras, Actors, Static Meshes, and so on.
5	Outliner	Displays a hierarchical tree view of all content in your Level.
6	Details panel	Appears when you select an Actor. Displays various properties for that Actor, such as its **Transform** (position in the Level), Static Mesh, Material, and physics settings. This panel displays different settings depending on what you select in the Level Viewport.
7	Content Drawer	Opens the Content Drawer, a temporary Content Browser window, from which you can access all of the Assets in your Project.
8	Bottom Toolbar	Contains shortcuts to the Command Console, Output Log, and Derived Data functionality. Also displays source control status.

The **Main Toolbar** contains shortcuts to some of the most used tools and commands in Unreal Editor. It is divided into the following areas:

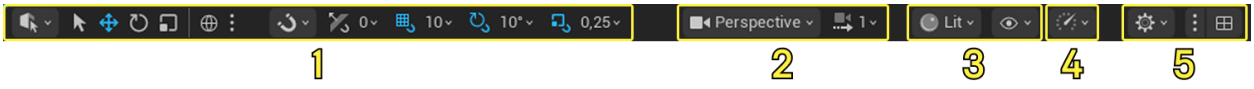


Number	Name	Description
1	Save button	Click this button to save the level that is currently open.
2	Select the open level	Click this button to show the currently loaded level in the Content Browser recently used Content Browser, which can be the docked Content Drawer Browser panel.
3	Mode selection	Contains shortcuts for quickly switching between different modes to edit c These modes also change the primary behavior of the Level Editor, which designed for each mode specifically. You can click the links to go to their more about a mode. • Selection • Landscape • Foliage • Mesh Paint • Modeling • Fracture • Brush • Animation
4	Content shortcuts	Contains shortcuts for adding and opening common types of content within

Number	Name	Description
		1. The **Create** button can be used to choose from a list of common **Act** **Used** assets from the Content Browser to quickly add to your level. 2. The **Blueprints** button can be used to create and access blueprints. 3. The **Cinematics** button can be used to create and access a **Level Seq** **Sequence** cinematic.
5	Play mode controls	Contains the shortcut buttons Play, Skip, Stop, and Eject for running your
6	Platforms menu	Contains a series of options you can use to configure, prepare, and deploy platforms, such as desktop platforms such as Windows, macOS and Linux, Some platforms, such as mobile and console, require additional setup and project. These can include their own software development kit (SDK) insta agreements (NDAs) to access them.

Viewport Toolbar

The Viewport Toolbar can be used to accomplish a number of tasks when building levels. It is located at the top of the Level Viewport, and its settings and tools are grouped into the following categories:



Number	Name	Description
1	Transform and Snapping tools	These are the options to swap between different Transform tools options.
2	Camera tools	These are the options to switch between Perspective and Orthogr camera movement speed.
3	View Mode and Show Flag options	These are the options to pick between different view modes like and show flags related to the current viewport to hide and reveal Level Viewport.
4	Performance and Scalability tools	This dropdown can be used to reveal settings like Viewport Scala Level.
5	Viewport options	These options can be used to change viewport related settings, lik Level Viewport.

To learn more about this toolbar, read the Viewport Toolbar documentation.

Personal Understanding

Game World Sections

Cell

Game worlds are purposefully divided into chunks for manageability upon load.
These chunks are called Levels/Cells/Worlds in Game Engine system view.
From open world games to linear ones they all use this system but in open world ones typically the world loading should happen by more estimations and according to variables such as draw distance and player location to keep the open world from being loaded late.

Level/UMap

A saved definition of a world section.

UWorld

A runtime simulation instance created from one or more umap defined levels.

Outliner

It represents and loads the engines **engine specific** filesystem accordingly.

Material Editor

Serves the same purpose as in UDK with some minor differences in UI**.**

Nodes

Each node could be from:

1- **The material function library:**
2- **Material Expression:**

Either way the input pins of a node are on the **Left** side, with the output pins on the **Right** side.

Material Output Nodes

These are the final interface between graph and the GPU shader result. Everything built in graph must ultimately be plugged into them. It defines the values the generated shader must compute.

- Every material must have ONE single output node.

- The output node is always there by default with a pretty descriptive name like Material or PreviewMaterial_2.

- The output node does not have an output pin cause… it IS the output pin itself.

It is worth mentioning that

1- Base Color: Takes color.
2- DiffusePower:
3- Emissive:
4- roughness:
5- Opacity:
6- OpacityMask:
7- Distortion:
8- TransmissionMask:
9- TransmissionColor:
10- Normal:
11- CustomLighting:
12- CustomLightingDiffuse:
13- AnisotropicDirection:
14- WorldPositionOffset: